AI-based anti-web attack

Prepared by:

Mouawia Fawaz Kaek

Supervised by:

Dr. Wassim Juneidi

Eng.Abdulrahman Arar

2026-2027

Table of contents

General Introduction 5
Background 5
Research Problem 6
Project Aim and Objectives 6
1. Guo et al. (2025) 7
2. Chen et al. (2025) 8
3. Bakır (2025) 8
4. Kareem et al. (2025) 8
5. Casmiry et al. (2025) 9
6. Wattanasuwan et al. (2025) 9
7. Rosca et al. (2025) 9
8. Urda et al. (2026) 10
9. Widyatama and Rahman (2025) 10
10. Song et al. (2026) 10
Comparative Summary of Previous Studies 11
Practical Applications 12
Scientific Challenges 13
Closing Paragraph 14
Chapter 1: 15
1.1 Chapter Introduction 16
1.2 Web Applications and Their Architecture 16
1.2.1 Definition of a Web Application 16
1.2.2 Core Components 16
1.2.3 The Client–Server Model 17
1.2.4 HTTP and HTTPS 17
1.2.5 Headers, Cookies, and Sessions 18
1.2.6 Application Programming Interfaces 18
1.3 Fundamentals of Web Application Security 18
1.3.1 Cybersecurity and Application Security 18
1.3.2 Information Security Properties 19
1.3.3 Assets, Threats, Vulnerabilities, Exploits, and Risks 19
1.3.4 Authentication and Authorization 19
1.3.5 Session Management 20

1.3.6 Input Validation and Processing 20
1.3.7 Security Logging and Monitoring 20
1.4 Relevant Web Attacks 21
1.4.1 Definition of a Web Attack 21
1.4.2 Injection 21
1.4.3 Cross-Site Scripting 21
1.4.4 Repeated Authentication Attempts 21
1.4.5 Resource Abuse and Service Disruption 22
1.4.6 Broken Access Control 22
1.4.7 Indicators Versus Confirmation of Compromise 22
1.5 Intrusion Detection Systems 22
1.5.1 Definition 22
1.5.2 Deployment and Data Sources 23
1.5.3 Detection Approaches 23
1.5.4 General Detection Pipeline 23
1.6 Artificial Intelligence and Machine Learning for Attack Detection 24
1.6.1 Basic Concepts 24
1.6.2 Supervised Learning 24
1.6.3 Unsupervised and Semi-Supervised Learning 25
1.6.4 Classification and Anomaly Detection 25
1.6.5 Common Machine-Learning Models 25
1.6.6 Benefits and Limitations of Machine Learning in Security 26
1.7 Data and Processing for Web-Attack Detection 26
1.7.1 Data Sources 26
1.7.2 Public Datasets 26
1.7.3 Data Cleaning and Validation 26
1.7.4 Feature Extraction and Representation 27
1.7.5 Data Splitting and Class Imbalance 27
1.8 Evaluating Detection Models 28
1.8.1 Confusion Matrix 28
1.8.2 Accuracy 28
1.8.3 Precision, Recall, and F1-Score 28
1.8.4 False-Positive Rate 28
1.8.5 Averages in Multiclass Classification 29
1.8.6 ROC and Precision–Recall Curves 29
1.8.7 Requirements for Fair Evaluation 29

1.9 Theoretical Limitations and Challenges 29
1.9.1 Data Quality and Representativeness 29
1.9.2 Generalization and Distribution Shift 30
1.9.3 False Alerts and Missed Attacks 30
1.9.4 Interpretability 30
1.9.5 Privacy and Legal Considerations 30
1.9.6 Limitations of Detection Systems 30
1.10 Chapter Summary 30

General Introduction

Background

The rapid development of information and communication technologies has made digital services central to individuals, businesses, and public institutions. Web applications now support a wide range of activities, including online education, commerce, banking, administration, communication, and data management. Their accessibility through browsers and mobile applications, together with their ability to serve many users and process large volumes of information, has contributed to their widespread adoption.

A typical web application consists of interacting components, such as a user interface, a web server, application logic, databases, and application programming interfaces. A user submits a request; the application processes it according to its defined rules, accesses or updates data when necessary, and returns a response. Although this architecture enables flexible and convenient services, it also exposes multiple points of interaction that may be abused when an application is not designed, implemented, configured, and maintained securely.

Web application security is therefore a significant part of cybersecurity. A vulnerability may allow an unauthorized party to access sensitive information, alter data, perform unauthorized operations, or disrupt a service. Security depends not only on the application's source code but also on authentication, authorization, session management, input validation, server configuration, error handling, security logging, traffic monitoring, and incident response.

The OWASP Top 10:2025 identifies several important categories of application-security risk, including broken access control, security misconfiguration, injection, authentication failures, and insecure design. It is an awareness document for developers and security practitioners, rather than an exhaustive catalogue of every possible attack or a substitute for a detailed security assessment. The OWASP Web Security Testing Guide provides a broader testing framework covering areas such as authentication, authorization, session management, input validation, error handling, and web services. Together, these resources illustrate that web security requires coordinated preventive, testing, monitoring, and response measures.

Traditional defensive mechanisms often rely on predefined rules or signatures. Such mechanisms can be effective when an attack pattern is already known and can be represented by a reliable rule. However, modified or previously unseen patterns may be harder to identify through fixed signatures alone. This limitation motivates the study of approaches that analyze data and behaviour to identify suspicious or anomalous activity.

Intrusion detection is concerned with monitoring events in a computer system or network and analyzing them for indications of possible security incidents. Intrusion detection and prevention systems may also record information, report incidents, or attempt to stop suspicious activity, depending on their design and deployment. Artificial intelligence and machine learning can support this analysis by processing data and learning patterns that may help distinguish ordinary activity from potentially malicious activity. A model may, for example, be trained using labelled data to classify new observations, or it may be used to identify deviations from expected behaviour.

Machine learning does not, by itself, guarantee that every attack will be detected. Results depend on the quality and relevance of the data, the reliability of its labels, the selected features, the

training process, and the evaluation design. Models may produce false positives by classifying legitimate activity as malicious, or false negatives by failing to identify an actual attack. Their output should therefore be evaluated using appropriate measures and treated as a component of a broader security strategy, not as a replacement for secure development practices or other defensive controls.

This project is based on the idea of studying and developing an artificial-intelligence-based intrusion detection system for web attack detection. The proposed system is intended to analyse data associated with web application activity or related communications and use relevant characteristics to help distinguish benign activity from activity that may indicate an attack.

Research Problem

Web applications receive requests and communications from users and networks that cannot always be assumed to be trustworthy. Some activity may be legitimate, while other activity may indicate attempts to exploit an application, bypass authentication, access unauthorized resources, submit malicious input, or abuse a service through repeated requests. Distinguishing between these cases can be difficult, especially when the volume of data is large, attack techniques change, or malicious requests resemble legitimate ones.

The problem is also shaped by the type of data available to the detection system. A system may analyse web-server logs, HTTP request content, network-flow characteristics, or another source. These sources are not interchangeable. A web-server log may record a request path, response code, and timestamp, whereas a network-flow record may describe connection duration, packet counts, or data volumes without exposing the contents of an individual HTTP request. Consequently, the selected data source determines which characteristics can be analysed and what conclusions the system can reasonably draw.

Machine-learning-based detection introduces additional methodological challenges. These include data quality, label reliability, class imbalance, feature selection, and possible leakage between training and test data. Overall accuracy alone may not adequately describe a detector's performance, particularly when benign samples substantially outnumber attack samples. Evaluation should therefore consider multiple measures, such as precision, recall, F1-score, and false-positive rate, and interpret them in relation to the data and intended use.

The scientific problem addressed by this project can be expressed as follows:

How can a system based on artificial intelligence and machine learning be designed to analyse data associated with web applications and support the detection of web attacks, while evaluating its ability to distinguish benign from suspicious activity and identifying its limitations?

Project Aim and Objectives

The project aims to study, design, and implement a prototype system for detecting web attacks using artificial intelligence and machine-learning techniques. The work will include a theoretical study of web application security and intrusion detection, an examination of relevant previous research, and the selection of an appropriate data source and attack scope, the proposed system or model will be developed and tested using suitable data, and its performance will be assessed using appropriate evaluation measures.

The project is based on the study *Machine learning-based intrusion detection system for detecting web attacks* by Fatimetou Abdou Vadhil, Mohamed Lemine Salihi, and Mohamedade Farouk Nanne, published in 2024 in the *IAES International Journal of Artificial Intelligence*. The study investigated machine-learning methods for identifying abnormal behaviour associated with web attacks. It used the CIC-IDS-2017 dataset and evaluated Decision Tree, Random Forest, Logistic Regression, Gaussian Naïve Bayes, Adaptive Boosting, and an ensemble approach. The authors assessed the models using accuracy, precision, recall, F1-score, false-positive rate, ROC curves, and calibration curves.

The present project adopts the general idea of using machine learning to analyse security-related data and support web attack detection. It does not assume that it will use the same dataset or algorithms, nor that it will reproduce the study's results. Those decisions depend on the methodology, data availability, and evaluation design approved for this project. The intended contribution is an applied workflow that proceeds from defining and preparing the data to selecting features and models, testing the system, analysing the results, and documenting its limitations.

The specific objectives are to:

1. Study the fundamental concepts of web application security and relevant risks and attacks.
2. Examine intrusion detection systems, their data sources, and their detection approaches.
3. Study the use of artificial intelligence and machine learning in security-data analysis.
4. Define an appropriate data source and detection scope.
5. Prepare and analyse the data and select features relevant to the detection task.
6. Design and implement a prototype that helps classify benign and suspicious activity within the selected scope.
7. Evaluate the prototype using appropriate performance measures and analyse false positives and missed attacks.
8. Document the findings, limitations, and challenges, and identify potential directions for future development.

Previous Studies

This section presents ten studies published in 2025 or 2026, excluding the 2024 study on which the project idea was based. The selected studies address web attack detection, SQL injection, or cross-site scripting (XSS), while using different data sources and analytical methods. They do not all examine the same detection task: some analyse HTTP requests or attack payloads, some process web-server logs, and others classify network traffic. Their results must therefore be interpreted within the specific datasets and experimental settings used in each study rather than compared as if they were obtained from a common benchmark.

1. Guo et al. (2025)

Guo et al. presented *Web-FTP: A Feature Transferring-Based Pre-Trained Model for Web Attack Detection* in *IEEE Transactions on Knowledge and Data Engineering*. The study addressed the difficulty of obtaining large, accurately labelled datasets for supervised web-attack detection. The authors proposed a pretrained model with preprocessing, pretraining, and deployment components. The approach learns general representations from unlabelled data and then transfers those representations to web-attack detection tasks. The authors reported improved classification

performance across several datasets and described deployment in real-world systems to demonstrate the approach's potential for industrial use.computer

This study is directly relevant because it addresses web-attack detection and examines the challenges of labelled-data scarcity and model deployment. It can inform the project's discussion of how a model might generalize across datasets and operating environments. However, pretrained feature-transfer methods are more complex than basic supervised classifiers.

2. Chen et al. (2025)

Chen et al. investigated SQL-injection detection in *Research on SQL Injection Detection Technology Based on Content Matching and Deep Learning*, published in *Computers, Materials & Continua*. The study focused on limitations in detecting varied forms of SQL injection using traditional methods. It proposed combining content matching and feature extraction with an improved convolutional neural network (CNN). The authors argued that some parsing and word-embedding procedures may conceal useful information or introduce noise, and therefore used preprocessing and content matching to preserve relevant characteristics. The experiments included benign, SQL-injection, and XSS data. The authors reported an accuracy of 98.35% in their experimental setting.

This study is relevant to the project because it illustrates how data preprocessing and representation may affect detection performance, in addition to the choice of classifier. It also addresses injection attacks that may target web applications. Its principal focus, however, is SQL injection, and its results depend on the specific data and model configuration used. They should not be generalized automatically to all web attacks or to a different data source.

3. Bakır (2025)

Rezan Bakır proposed *UniEmbed: A Novel Approach to Detect XSS and SQL Injection Attacks Leveraging Multiple Feature Fusion with Machine Learning Techniques* in the *Arabian Journal for Science and Engineering*. The study combined three text representations—Word2Vec, Universal Sentence Encoder (USE), and FastText—to capture word-level, contextual, and subword characteristics in web inputs. Several machine-learning classifiers were then evaluated on XSS and SQL-injection datasets. The study reported that the multilayer perceptron (MLP) achieved strong results on the tested datasets, with low false-positive and false-negative counts under the experimental setup.

The study is relevant because it examines feature representation for textual web inputs and covers two major classes of web attacks. It demonstrates that representation choices can influence a model's ability to distinguish benign and malicious samples. However, its text-embedding approach and datasets may not be suitable for network-flow data or server logs without further testing.

4. Kareem et al. (2025)

Kareem et al. examined machine-learning methods for web-attack detection in *Machine Learning-based Detection of Web Attacks Using Logistic Model Trees*, published in the proceedings of the *2025 International Conference on Computer Science and Software Engineering*. The study compared Smooth Private Forest and Logistic Model Trees using CICIDS2017. It reported strong results for both models across measures such as accuracy, precision, recall, and F1-score in its experimental setting. The authors also identified flow characteristics—including average outbound packet length and inter-arrival-time features—as influential in detection.

This study is closely related to the project because it applies machine learning to web-related attack detection and examines network-flow features. It can inform discussion of flow-based data and model comparison. However, the use of network-flow characteristics should not be described as direct analysis of the contents of every HTTP request unless the study's data explicitly supports that interpretation. The distinction between flow analysis and HTTP-payload analysis is also important when defining the present project's scope.

5. Casmiry et al. (2025)

Casmiry, Mduma, and Sinde studied SQL-injection detection in *Enhanced SQL Injection Detection Using Chi-Square Feature Selection and Machine Learning Classifiers*, published in *Frontiers in Big Data*. They combined a custom dataset with SQLiV3 data, represented queries using TF-IDF, and applied Chi-square feature selection before training five classifiers: multinomial Naïve Bayes, support vector machine (SVM), Logistic Regression, Decision Tree, and k-nearest neighbours (KNN). The study found that feature selection improved classification across the evaluated models by reducing noise and redundant features. After feature selection, the Decision Tree achieved the highest reported performance in that experiment, with 99.73% accuracy, 99.72% precision, 99.70% recall, and a 99.71% F1-score.

This study is relevant to the project's data-preparation and feature-selection considerations. It shows that model performance may change substantially after feature selection, and that model choice should be evaluated together with preprocessing. Its findings are specific to SQL-injection data and the datasets used; they do not establish that the same feature-selection procedure will work equally well for HTTP logs or network flows.

6. Wattanasuwan et al. (2025)

Wattanasuwan et al. proposed *A Dynamic XSS Attack Detection System with Realtime and Fast Response Based on Transformer Learning Model and Adaptive Rule-based System* at the *2025 Thirteenth International Symposium on Computing and Networking Workshops (CANDARW)*. The study focused on XSS detection and combined regular-expression-based rules for known patterns with a Transformer model based on CodeBERT for identifying other malicious inputs. It also proposed storing detected patterns and updating rules to accelerate the handling of recurring cases.

This work illustrates a hybrid approach that combines rule-based detection with machine learning rather than relying exclusively on either method. It is relevant to the project's discussion of XSS detection and operational response time. However, its Transformer model and adaptive rule-update mechanism are more complex than a basic classification system and should be treated as an alternative research approach, not as an assumed component of the project.

7. Rosca et al. (2025)

Rosca, Stancu, and Popescu examined SQL-injection detection in *Machine Learning Models for SQL Injection Detection*, published in *Electronics*. The authors assembled a dataset containing benign and malicious queries, applied normalization and feature-extraction steps, and compared multiple machine-learning models. They also assessed the impact of different sampling and modelling approaches. The study reported that an ensemble model achieved the strongest results in its

experimental setting. It discussed integrating a detection model as an additional security layer while retaining established controls such as parameterized queries and secure input handling.

This study is relevant because it describes a query-processing pipeline and emphasizes the importance of balancing attack detection with false alarms. It supports the view that machine learning should complement, rather than replace, established security controls. Its reported performance must be interpreted in light of the datasets and experimental configuration used.

8. Urda et al. (2026)

Urda et al. presented *Enhancing Web Traffic Attacks Identification Through Ensemble Methods and Feature Selection* in the *Logic Journal of the IGPL*. The study used the CSIC2010 v2 dataset, which contains simulated e-commerce HTTP traffic, to extract features from HTTP traces. It compared ensemble methods, including Random Forest and Extreme Gradient Boosting, with baseline classifiers such as k-nearest neighbours, LASSO, and SVM. The authors reported that the ensemble methods outperformed the baselines by approximately 20% in predictive accuracy and achieved an area under the ROC curve (AUC) of 0.989. They also evaluated feature-selection methods, including Information Gain, LASSO, and Random Forest.

This study is particularly relevant if the project uses HTTP traces or request-level information, because it examines web traffic rather than only general network flows. It also provides a useful comparison of ensemble classifiers and feature selection. The authors note limitations in the dataset, including its age and lack of HTTPS traffic, which should be considered when discussing how well results might transfer to contemporary applications.

9. Widyatama and Rahman (2025)

Widyatama and Rahman studied SQL-injection and XSS detection from Nginx server logs in *Analysis of SQL Injection and Cross-Site Scripting (XSS) Attacks on Web Server Logs Using Machine Learning*, published in the *Indonesian Journal of Artificial Intelligence and Data Mining*. The authors processed log data using regular expressions for parsing and labelling, producing 1,650,615 samples. They compared Random Forest and SVM and addressed class imbalance. The study reported that Random Forest achieved the best results in its experiment, including 99.92% accuracy, a 99.94% F1-score, and an AUC of 0.9994.

This study is relevant because it analyzes web-server logs, a source that may be suitable for operational monitoring depending on the information recorded. It also illustrates the handling of large datasets and class imbalance. The high reported scores should nevertheless be interpreted in the context of the log source, labelling procedure, and test design; they do not by themselves establish generalization to other servers or applications.

10. Song et al. (2026)

Song, Hong, and Kang proposed a web-attack detection system in *An Implementation of TF-IDF Feature Extraction and Machine Learning Based Web Attack Detection System*, published in the *Journal of The Korea Society of Computer and Information*. The authors treated HTTP requests as text, applied TF-IDF for feature extraction, and evaluated Logistic Regression, Random Forest, and XGBoost using the CSIC 2010 HTTP dataset. The study reported that XGBoost performed best among the three models, with 98.77% accuracy and an ROC AUC of 0.994; the other models also achieved accuracy above 97%.

This study is directly relevant because it analyses HTTP request data and describes a comparatively interpretable machine-learning pipeline. It can support discussion of text-based feature extraction and traditional classifiers as alternatives to more complex deep-learning models. Its results are tied to CSIC 2010, however, so dataset age and representativeness should be considered when relating the findings to current web applications.

Comparative Summary of Previous Studies

Article title	**Author(s)**	**Publisher**	**Year**	**Relevance to the project**
Web-FTP: A Feature Transferring-Based Pre-Trained Model for Web Attack Detection	Zhenyu Guo et al.	*IEEE Transactions on Knowledge and Data Engineering*	2025	Examines web-attack detection, pretrained representations, and the challenge of limited labelled data.
Research on SQL Injection Detection Technology Based on Content Matching and Deep Learning	Yuqi Chen, Guangjun Liang, and Qun Wang	*Computers, Materials & Continua*	2025	Studies preprocessing and CNN-based detection of SQL injection, with benign and XSS data included in the experiments.
UniEmbed: A Novel Approach to Detect XSS and SQL Injection Attacks Leveraging Multiple Feature Fusion with Machine Learning Techniques	Rezan Bakır	*Arabian Journal for Science and Engineering*	2025	Uses fused text representations for detecting XSS and SQL injection.
Machine Learning-based Detection of Web Attacks Using Logistic Model Trees	Mohammed Ibrahim Kareem et al.	Proceedings of CSASE 2025	2025	Compares models using CICIDS2017 and informs discussion of flow-based features.
Enhanced SQL Injection Detection Using Chi-Square Feature Selection and Machine Learning Classifiers	Emanuel Casmiry, Neema Mduma, and Ramadhani Sinde	*Frontiers in Big Data*	2025	Demonstrates the impact of feature selection and preprocessing on SQL-injection classification.

Article title	Author(s)	Publisher	Year	Relevance to the project
A Dynamic XSS Attack Detection System with Realtime and Fast Response Based on Transformer Learning Model and Adaptive Rule-based System	Kittipat Wattanasuwan et al.	Proceedings of CANDARW 2025	2025	Combines rule-based methods and a Transformer model for XSS detection.
Machine Learning Models for SQL Injection Detection	Cosmina-Mihaela Rosca, Adrian Stancu, and Catalin Popescu	*Electronics*	2025	Studies query preprocessing, feature extraction, and machine-learning models for SQL-injection detection.
Enhancing Web Traffic Attacks Identification Through Ensemble Methods and Feature Selection	Daniel Urda et al.	*Logic Journal of the IGPL*	2026	Uses HTTP traces and evaluates ensemble methods and feature selection.
Analysis of SQL Injection and Cross-Site Scripting (XSS) Attacks on Web Server Logs Using Machine Learning	Adi Septian Widyatama and Atep Aulia Rahman	*Indonesian Journal of Artificial Intelligence and Data Mining*	2025	Analyses Nginx logs and addresses class imbalance when detecting SQLi and XSS.
An Implementation of TF-IDF Feature Extraction and Machine Learning Based Web Attack Detection System	Eun ji Song, Seong-Cho Hong, and Ah Reum Kang	*Journal of The Korea Society of Computer and Information*	2026	Classifies HTTP requests using TF-IDF and traditional classifiers on CSIC 2010.

Practical Applications

The proposed system may support security monitoring in a web-application or laboratory environment by analysing a data source selected during the methodology phase, such as HTTP requests, web-server logs, or network-flow characteristics. Its practical purpose would be to classify

activity or highlight samples that warrant further investigation, rather than to make an independent and final security decision.

Potential applications include:

- Analysing web logs or requests to identify activity that may indicate an attack.
- Supporting the triage of security alerts and helping analysts prioritize events for review.
- Comparing machine-learning models using datasets with documented sources and labels.
- Presenting model outputs and evaluation measures through a report or interface, if included in the implementation.
- Providing a prototype that can be extended in future work to include additional data sources, attack categories, or monitoring mechanisms.

Completing a prototype does not, by itself, establish that the system is ready for production deployment. Operational use would require further testing on representative data, assessment of latency and resource consumption, analysis of false alarms, and clearly defined verification and response procedures.

Scientific Challenges

A primary challenge is defining the data source and detection scope. HTTP requests, web-server logs, and network flows contain different information and cannot be assumed to support the same detection conclusions. The project must clearly identify its unit of analysis, the data source, and whether the model classifies requests, log records, or flows.

Data quality, labelling, and representativeness are also important challenges. A dataset may contain duplicates, missing values, or imbalanced classes, and a laboratory dataset may not reflect the diversity of real-world traffic and attack behaviour. Evaluation scores may also be overstated if highly similar samples appear in both training and test sets or if information from the test set influences training. Data preparation and splitting must therefore be documented carefully.

Feature selection and model choice present further methodological challenges. Text-based characteristics may be appropriate for analysing requests and payloads, whereas timing, volume, and connection characteristics may be more suitable for network flows. A more complex model does not necessarily perform better than a simpler one; it may also require more resources and be harder to interpret. Comparisons should be based on documented and fair experimental settings.

The system must also balance the detection of attacks against the reduction of false alarms. A highly sensitive model may generate many false positives, while a less sensitive model may miss attacks. Precision, recall, F1-score, confusion matrices, false-positive rate, and—where appropriate—overall accuracy should therefore be considered together and interpreted in light of class distribution and security priorities.

Generalization and changing attack behaviour are additional concerns. A model may learn patterns specific to one dataset and perform less effectively when applied to a different application or network. Attack techniques also evolve, which may require models to be reassessed or updated. Finally, all practical security testing must be conducted in an authorized environment using lawfully

obtained data. The project is intended to address defensive analysis, detection, and evaluation, not to target real systems without permission.

Closing Paragraph

This project examines the use of artificial intelligence and machine learning to support the detection of web attacks. The introduction has presented the topic's significance, defined the research problem and objectives, and reviewed recent studies that use different models and data sources. The final selection of data, algorithms, attack categories, and evaluation procedures will be made as part of the approved methodology so that the completed report accurately reflects the system that is implemented and tested.

Chapter 1:
Theoretical Study

1.1 Chapter Introduction

This chapter presents the theoretical foundations required to understand the project on web-attack detection using artificial intelligence and machine learning. It first introduces the architecture of web applications and the exchange of data between clients and servers. It then discusses fundamental web application security principles and relevant attack categories, followed by an overview of intrusion detection systems and their principal detection approaches. Finally, it introduces machine learning concepts related to security-data analysis, including data preparation, feature selection, and model evaluation.

The chapter distinguishes general theoretical background from implementation decisions that have not yet been finalized. The project's data source, attack categories, algorithms, and implementation tools will be selected as part of the methodology. Accordingly, this chapter explains relevant concepts and alternatives without presenting an unapproved implementation choice as a project fact.

1.2 Web Applications and Their Architecture

1.2.1 Definition of a Web Application

A web application is a software system that provides functions or information to users through a browser or another client interface communicating with a server over a network. Its functionality may be as simple as displaying a page or as complex as managing accounts, processing transactions, storing data, and integrating with external services.

Unlike a static website, a web application may change its content or behaviour according to the user, the supplied input, or the current system state. For example, it may display a user-specific dashboard after authentication, search for a term supplied by the user, or store a submitted transaction in a database. Consequently, web applications process data from multiple sources and must validate and safely handle that data before using it.

1.2.2 Core Components

A typical web application consists of several interacting components:

- **Client:** The software through which a user accesses the application, usually a web browser or an application that communicates with web APIs. The client displays information and sends requests, but should not be treated as a trusted environment for enforcing sensitive security decisions.
- **Web server:** Receives HTTP or HTTPS connections and routes requests to the appropriate resources or application components. It may also serve static files, apply access rules, and record activity.
- **Application logic:** Implements business rules, such as validating an operation, processing a request, or enforcing a user policy.
- **Database:** Stores application information, such as user records, logs, and transactions, and supports read, insert, update, and delete operations.

- **Application programming interfaces (APIs):** Provide structured mechanisms for exchanging data between application components or with external services. They may be used by mobile applications, third-party services, or different parts of the same system.
- **Supporting services and infrastructure:** May include authentication services, caches, load balancers, message queues, logging systems, and monitoring tools.

These components do not necessarily run on one machine. They may be distributed across servers, containers, or cloud services. As the number of components and connections increases, so does the importance of managing permissions, securing communication channels, and monitoring events across the system.

1.2.3 The Client–Server Model

Web applications commonly use the client–server model. The interaction begins when the client sends a request to a server. The request identifies the required resource or operation and may include data in the URL, HTTP headers, or message body. The server receives and processes the request according to the application's rules and configuration, then returns a response containing a status code and, where appropriate, content or an operation result.

A general interaction can be described as follows:

1. The user selects an operation or resource in the application interface.
2. The client converts the operation into a network request, such as a GET request to retrieve a resource or a POST request to submit data.
3. The web server receives the request and routes it to the application logic.
4. The application validates the data, identity, and permissions as required.
5. The application communicates with a database or another service when necessary.
6. The server returns a response to the client, which may contain content, an error message, or the result of the operation.
7. Selected request and response details may be recorded for operational or security analysis.

This interaction is relevant to a web-attack detection project because it identifies where analysable data may be generated. The data source might be a server log containing request characteristics, an application log containing processing outcomes, or network-flow data describing a connection without containing the full request content.

1.2.4 HTTP and HTTPS

The Hypertext Transfer Protocol (HTTP) is used to exchange requests and responses between web clients and servers. A request typically contains an HTTP method, a resource path, the protocol version, headers, and, where appropriate, a body containing additional data. Common methods include GET, generally used to retrieve a resource, and POST, generally used to submit data. Other methods, such as PUT, DELETE, and PATCH, may be used according to application functionality.

A server response includes a status code indicating the result of processing, such as success, redirection, a client-side error, or a server-side error. It may also contain headers and a response

body. These details can be useful in web-log analysis. For example, repeated occurrences of a particular status code or relationships between paths and requests may be examined. However, an individual status code does not prove that an attack has occurred; it must be interpreted in context.

HTTPS is HTTP carried over Transport Layer Security (TLS). TLS is intended to protect communications against unauthorized reading or alteration while in transit and to support verification of the server's identity through certificates and trust configuration. HTTPS does not, by itself, make an application free of vulnerabilities. It protects the communication channel but does not automatically prevent SQL injection, XSS, or access-control errors.

1.2.5 Headers, Cookies, and Sessions

HTTP headers carry metadata that assists the processing of requests and responses. They may indicate content type, client type, content negotiation, authentication information, or routing details. Applications may also use cookies to carry a session identifier or selected preferences between a client and a server.

Because HTTP is stateless by default, many applications use sessions to associate consecutive requests with a particular user or application state. The server commonly retains session data and sends the client a unique identifier that is returned with subsequent requests. Session identifiers must be protected against guessing, theft, and unintended disclosure because possession of an identifier may enable session impersonation if other safeguards are absent. OWASP's session-management guidance describes measures for protecting session identifiers and managing session state securely.

From a detection perspective, logs may record session-related activity, repeated login attempts, or unusual changes in request patterns. Collecting and using such information should respect privacy requirements and minimize the storage of sensitive data. A cookie or session identifier in a request should not, on its own, be treated as evidence of an attack.

1.2.6 Application Programming Interfaces

Application programming interfaces (APIs) enable data and functions to be exchanged between application components or between an application and an external service. An API may accept data in formats such as JSON or XML and may use authentication and authorization mechanisms to verify the identity of a caller and the operations that caller is permitted to perform.

APIs form part of an application's attack surface because they accept input and may perform operations that affect data or system resources. Relevant risks include inadequate input validation, failure to check access to a specific object, excessive data exposure, and poorly configured rate limits. Detecting suspicious API activity depends on the available records, which may include the endpoint, request method, identity or role, request size, and request frequency. The handling of these records must also avoid unnecessary storage of secrets or personal data.

1.3 Fundamentals of Web Application Security

1.3.1 Cybersecurity and Application Security

Cybersecurity encompasses practices, technologies, and policies intended to protect systems, networks, and data against unauthorized access, malicious use, or disruption. Web application security focuses on protecting an application, its components, and its data throughout the

application lifecycle, including design, development, testing, deployment, operation, and maintenance.

Application security cannot be achieved by adding a single tool at the end of development. It requires integrating security requirements into design, coding, testing, and operation, while also managing updates, permissions, and logs. The OWASP Top 10:2025 identifies risks across multiple areas, including access control, security configuration, software supply chains, cryptography, injection, design, authentication, data integrity, security logging and alerting, and exceptional-condition handling.

1.3.2 Information Security Properties

Information protection is commonly described in terms of three related objectives:

- **Confidentiality:** Information should be accessible only to authorized people or systems. Unauthorized access to records or exposure of credentials may compromise confidentiality.
- **Integrity:** Data should remain accurate and should not be changed or destroyed without authorization. Unauthorized modification of records, transactions, or application settings can compromise integrity.
- **Availability:** Systems and data should remain accessible when needed. Resource exhaustion, service disruption, and system failures can threaten availability.

Other related properties include authenticity, accountability, and traceability. Security logging can help establish what happened, when it happened, and, where appropriate, which account or source was involved. Authentication supports verification of a user's or service's identity.

1.3.3 Assets, Threats, Vulnerabilities, Exploits, and Risks

Several related terms should be distinguished during security analysis:

- **Asset:** Something of value that requires protection, such as a user database or a login service.
- **Threat:** A condition, actor, or event that could cause harm to an asset.
- **Vulnerability:** A weakness in design, implementation, or configuration that may be exploited.
- **Exploit:** A method or action that takes advantage of a vulnerability to produce an unauthorized effect.
- **Risk:** An assessment of the likelihood that a threat will occur and the potential impact of that event.

An unusual request does not, by itself, prove that a compromise has occurred, and the existence of a vulnerability does not establish that it has been exploited. A detection system should therefore distinguish a potential indicator from a confirmed incident and, where possible, provide context or a confidence measure.

1.3.4 Authentication and Authorization

Authentication verifies the identity of a user or system, for example by checking a username and password or a security token. **Authorization** determines what an authenticated identity is allowed to access or perform. Authentication may succeed while an application remains vulnerable if it fails

to check whether the user is permitted to access a particular resource or perform a particular operation.

The principle of least privilege requires each user or component to receive only the permissions necessary for its work. Authorization checks should be performed by a trusted part of the application and should not rely solely on hiding interface elements or validating permissions in the client. Checks may need to consider the user's role, ownership of the requested record, and the context of the operation.

1.3.5 Session Management

Sessions allow an application to maintain a user's state across consecutive HTTP requests. Session identifiers should be difficult to guess and managed in a way that limits theft or reuse. Appropriate defensive measures include using HTTPS throughout the session, configuring suitable cookie attributes, renewing the identifier after sensitive events such as authentication, and invalidating the session after logout or expiry.

Sessions are relevant to detection when analysing login activity, device or network changes, or the sequence of requests within a session. These indicators do not establish an attack on their own: a user may legitimately change networks or devices. Detection requires contextual interpretation and appropriate privacy safeguards.

1.3.6 Input Validation and Processing

Web applications receive input from forms, URLs, headers, cookies, and APIs. The application should validate input according to its type and intended use, with validation performed on the server as the trusted enforcement point. Validation may check structure, type, length, range, and allowed values. It does not, however, replace safe database interfaces or context-aware output handling.

OWASP recommends parameterized queries to reduce the risk of user input being incorporated directly into SQL commands. Separating the query structure from its parameter values helps prevent input from changing the intended meaning of the query. When displaying untrusted data in a browser, OWASP recommends context-appropriate output encoding so that data is treated as data rather than executable content.

1.3.7 Security Logging and Monitoring

Event logs can support application monitoring, troubleshooting, and incident investigation. Depending on the application, they may record request times, paths, response codes, account identifiers or pseudonyms, authentication outcomes, and error details. Log collection should be carefully designed: logs must be protected against unauthorized access or modification, and passwords, secret tokens, and unnecessary sensitive data should not be stored in clear text.

Logging alone does not provide effective monitoring. Events must be sufficiently structured for analysis, related events should be correlated where appropriate, and alerts should be reviewed through an established response process. OWASP's 2025 risk list includes failures in security logging and alerting, highlighting how limited monitoring can delay incident detection or impede investigation.

1.4 Relevant Web Attacks

1.4.1 Definition of a Web Attack

A web attack is an activity that targets a web application, web service, or one of its components by exploiting a weakness in design, implementation, configuration, or data and identity management. An attack may aim to disclose or modify data, perform unauthorized operations, impersonate a user, disrupt service, or bypass access restrictions.

This section describes attack categories relevant to the project topic. It does not imply that every category will be included in the implementation. The final categories depend on the selected data, its available labels, and the approved methodology.

1.4.2 Injection

Injection occurs when untrusted input is interpreted as part of a command, query, or executable expression. SQL injection is one example: user input influences a database query. Other injection types may affect different interpreters or services. OWASP identifies the use of user input to construct SQL queries without safe handling as a potential source of SQL-injection vulnerabilities. Core preventive measures include parameterized queries, avoiding query construction through string concatenation, and validating input in context.

For detection, a request or log associated with injection may contain unusual textual or structural patterns. Searching only for particular words can produce errors: the same word may appear in benign content, while an attacker may alter the payload's form. Detection performance therefore depends on the data and feature set and should not be reduced to matching a single string.

1.4.3 Cross-Site Scripting

Cross-site scripting (XSS) occurs when an attacker causes a web application to present untrusted input in a way that makes a user's browser interpret it as executable content. This may result from inadequate output encoding, unsafe input handling, or insecure use of browser-side interfaces.

XSS is commonly described as stored XSS, in which malicious content is stored and later presented to other users; reflected XSS, in which the application returns content associated with a request; and DOM-based XSS, in which unsafe client-side processing contributes to execution. A core defensive measure is context-appropriate output encoding, so that data is displayed as data rather than interpreted as code.

A detection system may analyse request text, recorded output, or behavioural indicators, but its ability to observe the payload depends on its data source. A web log may contain a path and selected parameters without recording the complete request body; encryption may also limit visibility at some monitoring points. The data source must therefore be described before claiming that a system can detect every form of XSS.

1.4.4 Repeated Authentication Attempts

An attacker may attempt to guess credentials by submitting many login attempts, trying lists of passwords, or targeting multiple accounts. Brute-force attempts and password spraying are examples of such activity; their patterns differ according to rate, source distribution, and the accounts targeted.

Potential indicators include the number of login attempts within a time interval, the proportion of failed attempts, the number of accounts targeted, and changes in connection sources. These indicators warrant analysis but do not prove an attack: repeated failures may also result from a user who has forgotten a password or from a malfunctioning client. Alerts should be assessed in the context of the application and its policies.

1.4.5 Resource Abuse and Service Disruption

A party may submit a large number of requests or consume processing, memory, or connection resources in a way that affects the application's ability to serve users. This kind of resource abuse differs from exploiting a specific software vulnerability and may relate to application design, rate limits, and resource-management controls.

Request rates, response sizes, processing times, and connection counts may help identify unusual activity. However, legitimate demand can rise during periods of high use. Detection should therefore rely on an appropriate baseline and context rather than an unjustified fixed threshold.

1.4.6 Broken Access Control

Broken access control occurs when an application fails to enforce appropriate restrictions on what a user or service may access. A user may view another user's information or perform an unauthorized operation if the application checks only that the user is logged in and does not verify access to the specific requested resource. OWASP identifies broken access control as a major web application risk in its 2025 Top 10.

Detection can be difficult when the data does not record the user's identity, role, requested resource, and expected operation context. Network characteristics alone may therefore be insufficient to detect some access-control failures; relevant application-level records may be required.

1.4.7 Indicators Versus Confirmation of Compromise

A system may identify a pattern resembling an attack or a deviation from expected behaviour, but that alert does not necessarily confirm a compromise. It is an indicator requiring interpretation and investigation. The activity may be legitimate, or the alert may result from a data or configuration issue. Where possible, detection systems should provide the type of indicator or a confidence measure and retain sufficient context for verification without exposing sensitive information.

1.5 Intrusion Detection Systems

1.5.1 Definition

An intrusion detection system (IDS) monitors events or data on a host or network and analyses them for activity that may represent security incidents or policy violations. Its output may be an alert, record, classification, or report that supports investigation.

An intrusion prevention system (IPS) generally adds the ability to take action to block or limit activity, such as dropping a connection or blocking a request, according to its design and policy. Some products combine detection and prevention, but detecting activity and actively blocking it are distinct functions. NIST describes multiple IDPS technology classes, including network-based systems, wireless systems, network behaviour analysis systems, and host-based systems.

1.5.2 Deployment and Data Sources

Detection systems may be categorized according to the data they monitor or their deployment location:

- **Host-based IDS (HIDS):** Monitors events on a particular server or device, such as system logs, files, and processes.
- **Network-based IDS (NIDS):** Analyses network traffic observed at a monitoring interface or network point.
- **Network behaviour analysis (NBA):** Focuses on traffic patterns and changes in connection or flow characteristics.
- **Application-log analysis:** Examines web-server or application logs to analyse requests and processing outcomes.
- **Security information and event management (SIEM):** Collects and correlates data from multiple sources to support alerting, investigation, and search.

Deployment location affects the information available. A host-based system may observe internal details that are not visible in a network flow, whereas a network sensor may monitor communications from several devices without access to the application's internal logic. The detection location should therefore match the project's objective and available data.

1.5.3 Detection Approaches

Intrusion detection systems use several principal approaches:

1. **Signature-based or misuse detection:** Compares events against known patterns or rules. It can be effective when an attack pattern is known and well defined, but may fail to detect new, modified, or obfuscated attacks.
2. **Anomaly-based detection:** Establishes a representation of expected behaviour and identifies significant deviations. It may help identify previously unseen behaviour, but can generate false positives when legitimate activity is diverse or changes over time.
3. **Stateful protocol analysis:** Checks whether an interaction conforms to expected protocol behaviour and state. It may identify violations that simple text matching misses, but depends on accurate protocol models and sufficient context.
4. **Hybrid detection:** Combines approaches, such as signatures and anomaly analysis, or rules and machine-learning models, to use their complementary strengths and mitigate some limitations.

No single approach is ideal in every setting. Selection depends on the attack type, data source, available resources, and the acceptable cost of false positives and missed detections.

1.5.4 General Detection Pipeline

A general intrusion-detection pipeline may include the following stages:

1. **Data collection:** Capture logs, events, packets, or flows from the selected source.

2. **Preprocessing:** Normalize formats and handle missing or invalid values; remove duplicates where justified.
3. **Feature extraction:** Convert raw data into an analysable representation, such as request, connection, or timing characteristics.
4. **Analysis or classification:** Apply rules, statistical methods, or machine-learning algorithms.
5. **Output generation:** Classify an event, assign an anomaly score, or produce an alert.
6. **Verification and response:** Review the alert and take action according to policy and operational procedures.
7. **Evaluation and improvement:** Analyse errors and alerts and update data, rules, or models when appropriate.

An operational system also requires data-quality monitoring, log protection, model-update management, and change documentation so that it does not become a source of misleading information or an additional security weakness.

1.6 Artificial Intelligence and Machine Learning for Attack Detection

1.6.1 Basic Concepts

Artificial intelligence is a field concerned with designing systems that can perform tasks associated with capabilities such as perception, reasoning, or decision-making. Machine learning is a branch of AI in which models learn patterns from data for tasks such as prediction, classification, clustering, or anomaly detection.

In attack detection, a model may learn relationships between data features and known labels, such as “benign” and “attack,” and then apply what it has learned to new samples. Its output remains dependent on the distribution and preparation of its training data. A model should not be interpreted as possessing general knowledge of every attack; it learns patterns represented in the data and experimental design.

1.6.2 Supervised Learning

In supervised learning, each training sample has a known label or class, such as benign activity or a particular attack type. The model learns the relationship between features and labels from the training data and is then evaluated on samples not used for training.

The task may be framed as binary classification, for example:

- Benign.
- Attack.

It may instead be framed as multiclass classification, for example:

- Benign.
- SQL injection.
- XSS.

- Brute force.
- Other classes represented in the selected dataset.

Multiclass classification requires sufficiently representative and reliably labelled examples for each class. Performance should also be examined class by class, because an overall average may conceal weak performance on a rare attack category.

1.6.3 Unsupervised and Semi-Supervised Learning

Unsupervised learning uses data without labels for every sample and searches for structure, groups, or unusual observations. Clustering and anomaly-detection methods are examples. Such methods may help identify behaviour not represented in labelled attack data, but labelling a sample as anomalous does not prove that it is malicious; the result may reflect new legitimate behaviour or a data-collection issue.

Semi-supervised learning uses a limited amount of labelled data together with unlabelled data. It may be useful when labelling is costly, but it requires careful design to prevent early labelling errors from being reinforced by the model.

1.6.4 Classification and Anomaly Detection

Classification predicts a class from a predefined set of classes. Anomaly detection instead focuses on observations that differ from a reference pattern. A web-attack detection project may use either approach or combine them, but the choice depends on the availability of labels, the data, and the intended objective.

Supervised classification is suitable when labelled data cover the target classes. Anomaly detection may be useful when examples of some attacks are scarce or when the objective is to flag unusual behaviour. Both approaches require reliable data and careful examination of errors.

1.6.5 Common Machine-Learning Models

Models used for security classification include:

- **Logistic Regression:** A statistical classification model that can provide an understandable baseline and is often easier to interpret than more complex models.
- **Decision Tree:** Partitions data using conditions on features and produces decision paths that can be inspected, although it may overfit if not controlled.
- **Random Forest:** Combines multiple decision trees and can reduce variance and improve stability across many tasks.
- **Support Vector Machine (SVM):** Finds a decision boundary between classes and may be suitable for high-dimensional data, depending on preprocessing and data size.
- **Naïve Bayes:** Uses a simplified probabilistic model of the relationship between features and classes and is sometimes used as a fast baseline.
- **Gradient-boosting methods, such as XGBoost:** Build models sequentially to improve on previous errors and may perform well on structured data, subject to suitable parameter selection.

- **Neural networks and deep-learning models:** Include CNN, LSTM, and Transformer architectures and may suit text or sequential data, but require appropriate data, resources, tuning, and evaluation.

There is no universally "best" algorithm without testing it fairly on the intended data. A model should not be selected solely because it achieved a high score in another study: datasets, feature sets, data splits, and evaluation measures may differ.

1.6.6 Benefits and Limitations of Machine Learning in Security

Machine learning can help analyse numerous features and relationships, automate some forms of data triage, and identify patterns that may not be apparent from a single indicator. It may also support analysis of previously unseen combinations of characteristics.

However, models can learn biases in their training data, depend on unstable features, or be affected by changes in the data distribution. Some models are also difficult to interpret and may be deliberately evaded by carefully crafted inputs. NIST's AI Risk Management Framework identifies characteristics of trustworthy AI systems that include validity and reliability, safety, security and resilience, accountability and transparency, explainability and interpretability, privacy enhancement, and management of harmful bias. A model should therefore be evaluated in the context of its intended use, and its assumptions and limitations should be documented.

1.7 Data and Processing for Web-Attack Detection

1.7.1 Data Sources

A web-attack detection system may use one or more of the following sources:

- **Web-server logs:** Commonly record request time, path, HTTP method, response code, response size, and selected client details, depending on the logging configuration.
- **Application logs:** May record authentication outcomes, operations, application errors, or access to resources.
- **HTTP requests or payloads:** May provide textual or structural information about requests when that information is available and authorized for analysis.
- **Network flows:** Summarize connection properties, such as packet counts, data volume, duration, and direction, and do not necessarily include application content.
- **Packet captures (PCAP):** May provide detailed packet data, although content visibility depends on encryption, capture location, and authorization.

1.7.2 Public Datasets

CIC-IDS2017 is a publicly available intrusion-detection dataset published by the Canadian Institute for Cybersecurity. It includes benign traffic and several attack types, and provides labelled flow records generated using CICFlowMeter, as well as PCAP and CSV files. The official dataset description includes multiple protocols, normal background traffic, and several attack scenarios, including web attacks such as web brute force, XSS, and SQL injection.

1.7.3 Data Cleaning and Validation

Preprocessing may include the following steps:

- Standardizing column names, types, and formats.
- Handling missing or invalid values.
- Identifying duplicate records and making a documented decision about their treatment.
- Checking label definitions and consistency.
- Examining outliers and determining whether they are errors or meaningful cases.
- Removing identifiers that could reveal device identity or allow the model to memorize the experimental environment rather than learn generalizable behaviour.
- Protecting or anonymizing sensitive data where necessary.

Preprocessing must be designed to avoid test-data leakage. For example, if normalization statistics or feature-selection criteria are calculated using the complete dataset before splitting, the training process may use information that would not be available in a proper test setting. Such transformations should therefore be fitted on training data and then applied consistently to validation and test data.

1.7.4 Feature Extraction and Representation

Features are the characteristics used to represent samples to a model. Network-flow data may include duration, packet counts, byte rates, protocol, and inter-arrival times. Web-server logs may provide the HTTP method, request path, response code, and request counts over a time interval. Textual input may be represented using TF-IDF, n-grams, or language representations, depending on the task.

Features should be relevant to the detection objective and available in the intended use case. Care is also needed to avoid features that reveal the answer directly, such as a field that encodes the attack label or is derived from it. The report should describe and justify feature selection and, where appropriate, evaluate whether reducing the feature set improves performance, speed, or interpretability.

1.7.5 Data Splitting and Class Imbalance

Data are commonly divided into training, validation, and test sets. Training data are used to fit the model; validation data support parameter and model selection; and the test set provides a final assessment using samples that were not used for training or tuning.

The splitting strategy should reflect the structure of the data. If highly similar or temporally related samples are randomly distributed across all sets, testing may become easier than real deployment. Depending on the data, a time-based or source-based split may be more appropriate. Class imbalance occurs when the numbers of benign samples and different attack categories vary substantially. Resampling, class weighting, or suitable evaluation measures may be considered, but any training-time resampling must be confined to the training process so that the final evaluation remains valid.

1.8 Evaluating Detection Models

1.8.1 Confusion Matrix

A confusion matrix compares actual classes with predicted classes and helps explain the types of errors. In binary classification, the following terms are used:

- **True Positive (TP):** An attack sample correctly classified as an attack.
- **False Positive (FP):** A benign sample incorrectly classified as an attack.
- **True Negative (TN):** A benign sample correctly classified as benign.
- **False Negative (FN):** An attack sample incorrectly classified as benign.

These values show why detection errors are not equivalent in operational terms. A high FP count may overwhelm analysts with incorrect alerts, whereas a high FN count may allow actual attacks to remain undetected.

1.8.2 Accuracy

Accuracy measures the proportion of all predictions that are correct:

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}$$

Accuracy provides an overall measure but may be misleading when the classes are imbalanced. If most samples are benign, a model that always predicts “benign” may obtain high accuracy while failing to detect attacks.

1.8.3 Precision, Recall, and F1-Score

Precision measures the proportion of samples predicted as attacks that are actual attacks:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

Recall measures the proportion of actual attacks detected by the model:

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

The **F1-score** is the harmonic mean of precision and recall:

$$\text{F1} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

These measures help assess the balance between limiting false alerts and detecting attacks. The scikit-learn documentation describes precision as the ability not to label a negative sample as positive, recall as the ability to find positive samples, and F1 as their harmonic mean.

1.8.4 False-Positive Rate

The false-positive rate (FPR) measures the proportion of benign samples incorrectly classified as attacks:

$$\text{FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}}$$

This measure is important in operational systems because a high FPR may create a large number of unhelpful alerts and increase review workload. It should be considered alongside recall, since reducing false alarms can sometimes also reduce the number of attacks detected.

1.8.5 Averages in Multiclass Classification

When several attack categories are classified, measures may be calculated for each class and summarized using different averaging methods:

- **Macro average:** Calculates the measure for each class and gives each class equal weight, making it useful for highlighting rare classes.
- **Weighted average:** Weights each class result according to the number of samples in that class.
- **Micro average:** Aggregates the class-level counts before calculating the measure, so the result reflects the overall number of samples.

The averaging method should be stated because the resulting values can differ significantly when classes are imbalanced. Per-class results should be reported alongside aggregate measures whenever possible.

1.8.6 ROC and Precision–Recall Curves

A receiver operating characteristic (ROC) curve compares the true-positive rate with the false-positive rate at different classification thresholds. The area under the ROC curve (AUC) summarizes the model's ability to rank positive samples above negative ones across thresholds. A precision–recall curve focuses on the relationship between precision and recall and can be particularly useful when positive samples are rare.

AUC alone should not be treated as sufficient evidence of detection quality. It should be reported alongside the confusion matrix, other measures, and information about the data distribution and test design.

1.8.7 Requirements for Fair Evaluation

A sound evaluation requires preventing data leakage, choosing an appropriate splitting method, using training and validation data for model selection, and reserving the test set for final assessment. Experimental settings, software versions, preprocessing methods, data sources, labels, and evaluation measures should be documented.

Results from different papers should not be compared directly when the studies use different datasets, splits, class definitions, or evaluation procedures. A score of 99% on one dataset does not establish that the same model will achieve an equivalent score on another. Reproducibility and meaningful interpretation require a clear description of experimental conditions and limitations.

1.9 Theoretical Limitations and Challenges

1.9.1 Data Quality and Representativeness

Models depend on the data from which they learn. If the data do not represent the diversity of requests or the intended application environment, the model may learn narrow or temporary patterns. Results may also be affected by incorrect labels, duplicated records, missing values, or substantial class imbalance.

1.9.2 Generalization and Distribution Shift

Real-world data may differ from training data because of changes in applications, users, protocols, usage patterns, or attack types. This distribution shift can reduce model performance when it is transferred to a new environment. Results should not be generalized to environments that were not evaluated.

1.9.3 False Alerts and Missed Attacks

Balancing false positives against missed attacks is both a security and operational challenge. False positives consume analyst time and may reduce trust in a system, while missed attacks may allow harmful activity to continue. The appropriate balance depends on the context and cost of errors; it cannot be determined by selecting a metric without considering the use case.

1.9.4 Interpretability

The decisions of some models, particularly deep or ensemble models, may be difficult to interpret. Security analysts may need to know which features contributed to an alert before deciding whether to investigate or respond. Interpretability should therefore be considered alongside performance and complexity.

1.9.5 Privacy and Legal Considerations

Logs and requests may contain personal information, session tokens, credentials, or other sensitive details. Data collection should be limited to what is necessary, sensitive information should be anonymized where possible, file access should be controlled, and applicable legal and institutional requirements should be followed. Practical tests should be performed in an authorized environment or with data approved for research use.

1.9.6 Limitations of Detection Systems

An alert alone does not prove that a compromise has occurred, and a machine-learning model cannot guarantee complete protection against attacks. A system can only analyse the data available to it and may not detect activity that is not recorded or visible at its monitoring point. It should therefore be treated as a tool for supporting detection and analysis within a defence-in-depth strategy that includes secure development, access management, updates, logging, and incident response.

1.10 Chapter Summary

This chapter has presented the theoretical foundations required to understand the project on web-attack detection using artificial intelligence and machine learning. It began by describing web application architecture and client–server interaction, then discussed application-security fundamentals and the concepts of threats, vulnerabilities, and risks. It also described relevant attack categories, including injection, XSS, repeated authentication attempts, and broken access control.

The chapter then examined intrusion detection systems and signature-based, anomaly-based, protocol-analysis, and hybrid approaches. It introduced machine-learning concepts, potential data sources, data preparation, feature extraction, and model-evaluation measures. Particular attention was given to the need for multiple evaluation measures, prevention of data leakage, and discussion of limitations relating to generalization, privacy, and false alerts.

These theoretical foundations will inform the design of the proposed solution and the methodology presented in subsequent chapters .

Chapter 2

Design of the proposed solution